Mask-Proof: An LLM-based Automated Data Curation Pipeline on Mathematical Proofs

Jierui Zhang
jerryzhang@bupt.edu.cn
Beijing University of Posts and Telecommunications
Beijing, China

Siyuan Tan
cream_milk@bupt.edu.cn
Beijing University of Posts and Telecommunications
Beijing, China

Abstract

Large language models (LLMs) are increasingly capable of mathematical problem solving and can even assist with research-level proofs, yet we still lack a scalable and reproducible way to measure step-level reasoning in long proofs across diverse sources. This evaluation gap limits trustworthy AI assistance in proof-certified scientific progress. Existing evaluations often emphasize final answers or rely on costly expert grading, while end-to-end proof generation remains open-ended and hard to verify automatically. We introduce **Mask-Proof**, a pipeline that turns real proofs into automatically checkable masked-step tasks. It masks key formula steps, provides the necessary surrounding context, and evaluates model reconstructions with an LLM-based equivalence judge using repeated votes for stability. The resulting **Mask-ProofBench** contains 292 curated problems across diverse research areas. Experiments with 17 models show that reasoning-enhanced models outperform standard models by 12% to 27%. Our evaluator achieves 96.8% agreement with expert annotators, enabling faithful, reproducible, and comparable measurement of step-level mathematical reasoning. Benchmark, annotations, and code are available at https://anonymous.4open.science/r/MaskProof-Bench-7E47/.

Keywords

Mathematical Reasoning, Large Language Models, Benchmark, Proof Evaluation, Automated Curation

1 Introduction

Large language models (LLMs) are increasingly competent at mathematical problem solving [1] and can even assist with research-level proofs [24, 27]. Yet we still lack a scalable and reproducible measurement instrument for step-level reasoning in long-range proof—one that remains faithful to the underlying mathematics across heterogeneous sources such as research papers and competition benchmarks [18, 19, 31]. This gap is increasingly visible in recent frontier-system investigations. For instance, Sébastien Bubeck and collaborators at OpenAI report that GPT-5 can sometimes make meaningful progress on research-level proof tasks under expert guidance [2]. However, such evidence often relies on stepwise expert judgment, which is costly to scale, difficult to reproduce, and hard to compare across domains.

Why This Matters for AI for Sciences. This measurement gap is not merely a benchmarking nuisance. Mathematics is a foundational scientific discipline in which progress is certified by proofs. Accordingly, trustworthy AI assistance in mathematics requires domain-grounded evaluation of reasoning processes, not only final outcomes [4, 18, 28]. Mask-Proof contributes proof-centric data and evaluation tools that make step-level reasoning measurable, reproducible, and comparable across real-world proof sources.

The Measurement Problem in Step-Level Proof Evaluation. Treating step-level evaluation as a scientific measurement reveals three intertwined obstacles: what to measure, what context is required, and how to score correctness. (1) *Step selection is non-trivial.* Proofs vary widely in style and granularity. Many lines are trivial, purely definitional, or dominated by local surface cues. Masking such steps risks measuring shortcut exploitation rather than genuine inference, undermining construct validity [7, 29, 34]. (2) *Dependencies are often missing.* Research proofs frequently rely on external dependencies—cross-referenced lemmas, macros, or unstated background facts. If these dependencies are not recovered, a masked step may be underdetermined from the provided context, confounding reasoning ability with missing information and making failures uninterpretable. (3) *Scalable checking is difficult.* Even when a ground-truth step is available, correctness checking at scale remains hard: string matching is inadequate, symbolic tools offer no general guarantees, and expert evaluation does not scale. Automated judging therefore requires explicit mechanisms to control variance and ensure reproducibility [12, 13, 23, 33]. Taken together, these challenges prevent step-level reasoning in mathematics from being studied as a reproducible scientific phenomenon.

Mask-Proof: Making Step-Level Reasoning Measurable. We introduce **Mask-Proof**, a pipeline that directly addresses the obstacles above by transforming heterogeneous proofs into verifiable, self-contained, step-level evaluation problems with variance-controlled automatic checking. The core idea is intentionally simple: rather than scoring entire proofs end-to-end, we test whether a model can reconstruct a key formula step whose ground truth is extracted directly from the original proof. Concretely, Mask-Proof (i) uses Codex CLI to agentically select key steps that cannot be solved by local cues alone, (ii) recovers the minimal dependency closure needed to make each masked step self-contained, meaning that no external lemmas, definitions, or macros are further required beyond the provided context, and (iii) checks model outputs using a calibrated LLM-based equivalence judge, with repeated independent judgments and majority voting to reduce variance. Beyond benchmarking, Mask-Proof serves as a reusable measurement framework, exhibiting effectiveness, reproducibility, and generalization across proof sources. Figure 1 illustrates examples of the resulting masked-step format.

In this work, we make three contributions:

(1) **Mask-Proof**, an LLM-based automated curation pipeline that identifies inference-critical formula steps, reconstructs

self-contained proof context, and produces verifiable masked-step problems from real-world mathematical proofs.
(2) **Mask-ProofJudge**, an automatic step-level evaluation framework calibrated against expert judgments, achieving 96.8% human–judge agreement with variance reduction via repeated independent judgments and voting.
(3) **Mask-ProofBench**, a benchmark generated by this pipeline that enables scalable and reliable evaluation of step-level reasoning in real-world research-level mathematical proofs.

2 Related Work

Answer-based mathematical reasoning benchmarks. Evaluation of LLM mathematical reasoning has historically centered on benchmarks with uniquely verifiable endpoints. GSM8K [5] and MATH [10] established final-answer accuracy as the dominant metric, enabling scalable evaluation and providing limited visibility into reasoning processes. As performance on these benchmarks saturated, subsequent work increased difficulty through olympiad-level problems [6, 9], expert-authored frontier evaluations [8], and research-level question-answering [30]. However, even sophisticated answer-based benchmarks fundamentally evaluate *conclusions* rather than *formulas*, leaving proof quality unexamined.

Proof-centric evaluation and formal theorem proving. Proof-centric evaluation shifts focus from answer correctness to formula rigor, making verifiability the core challenge. Formal theorem proving addresses this by delegating correctness to proof assistant kernels: miniF2F [32] and PutnamBench [25] benchmark formal proving on competition mathematics. However, the natural-language-to-formal gap introduces semantic errors that are difficult to eliminate; ReForm [3] reveals that autoformalization is inherently challenging, with even human experts producing semantic errors in 38.5% of cases. Natural-language proof evaluation is closer to real mathematical practice but substantially harder to grade at scale. IMO-ProofBench [17] introduces rubric-based grading for competition proofs, while STORM-BORN [15] curates challenging formulas from research papers with human-in-the-loop verification. More recently, IMProofBench [20] targets *research-level* proof generation with expert-authored problems, and combines expert grading of full proofs with gradable follow-up subquestions. Yet these evaluations still require substantial human expert annotation and can't scale to evaluate in-the-wild research-level proofs.

Verifiers and scalable checking. To reduce reliance on expert grading, an important line of work trains verifiers that score reasoning processes. Process reward models [14, 16] provide step-level feedback, while diagnostic studies reveal that LLMs struggle to locate reasoning errors but can correct them given error locations [26]. Self-correction approaches have shown limited success: Huang et al. [11] demonstrate that LLMs cannot reliably self-correct reasoning without external feedback. More recent generator–verifier frameworks, such as DeepSeekMath-V2 [22], scale verification through meta-verification and self-training, though their evaluation remains centered on competition-style problems. High-profile demonstrations of AI-assisted mathematics [2] still emphasize expert oversight rather than fully automatic evaluation. Collectively, prior work highlights a persistent capability–verifiability tension: as evaluations approach research-level proofs, obtaining low-cost, scalable, and reproducible verification becomes substantially harder.

Table 1: Stage-wise sample counts in the Mask-Proof curation pipeline.

Step	# Usable Samples
Retrieve papers	08/2025–10/2025 (403 papers)
Extract proofs	835 proofs
Total Masks	292 masks

3 Methodology

In this section, we describe the overall pipeline, Mask-Proof, including automated paper collection and proof quality control, automated proof data curation and evaluation. This LLM-based pipeline is explicitly designed to be contamination-resistant by collecting and curating recent published papers, reproducible by using powerful Codex CLI, and broadly generalizable by adding different sources of proof data, while ensuring that the resulting masked proof data supports fully automated verification and maintains rigorous guarantees of correctness and verifiability. An overview of the pipeline is shown in Figure 2.

3.1 Paper Collection and Proof Extraction

To ensure the quality of the collected arXiv papers, proofs, and the resulting masked proof data, we curate a corpus of real-world, research-level mathematical papers under the following criteria. First, we restrict our corpus to arXiv submissions between August and October 2025, so as to minimize potential contamination from pretraining data and reduce the risk of data leakage in large language models. Second, we collect papers across a broad range of mathematical fields to mitigate domain bias; the distribution of the major subject categories is summarized in Table 6.

We mine these research-level papers together with their full LaTeX sources to enable precise proof extraction and dependency resolution. To ensure that the resulting evaluation instances require nontrivial reasoning, we further filter out trivial proofs, including very short arguments, purely definitional proofs, and pattern-completable formulas, as such instances either do not involve substantive inference or are prone to context leakage.

3.2 Automated Proof Data Curation

Self-contained Recovery. Formally, we define a proof to be self-contained if the pipeline can recover a sufficient dependency closure such that each masked step is solvable from the extracted theorem statement and the reconstructed proof context alone, with no reliance on external documents.

Concretely, the pipeline provides Codex CLI with (a) the proof snippet and (b) its source LaTeX data, and instructs it to resolve **explicit dependencies** by traversing internal cross-references with explicit labels (e.g., `\ref`, `\eqref`), locating the corresponding labeled targets in the source, appending the extracted content to the problem context, and expanding user-defined macros that carry essential semantics into standard LaTeX. In contrast, **implicit dependencies** are not directly anchored by labels (e.g., symbols

(I) IMO-ProofBench

Problem (PB-Advanced-029, USAMO 2025): Find all positive integers k such that $\frac{1}{n+1}\sum_{i=0}^{n}\binom{n}{i}^k$ is an integer for all $n \in \mathbb{Z}^+$.

Solution:
Answer: all even k. Let $S(n) := \sum_{i=0}^{n}\binom{n}{i}^k$.
Necessity. Choose $n = 2$. We need $3 \mid S(2) = 2 + 2^k$, which requires k to be even.
Sufficiency. From now on treat k as fixed and even. Let p^e be a prime power fully dividing $n+1$. It suffices to prove $S(n)$ is a multiple of p^e.
Lemma. Suppose p^e fully divides $n+1$. Then

$$\binom{n}{i} \equiv \pm\binom{\frac{n+1}{p}-1}{\lfloor i/p \rfloor} \pmod{p^e}$$

Proof. Write $\binom{n}{i} = \prod_{1\le j\le i}\frac{n-(j-1)}{j}$. The fractions for $p \nmid j$ are all $\pm 1 \pmod{p^e}$. So only consider those j with $p \mid j$; in that case one obtains the claimed binomial exactly.
From the lemma:

$$S(n) \equiv p \cdot S\left(\frac{n+1}{p} - 1\right) \pmod{p^e}$$

by grouping the $n+1$ terms into consecutive ranges of length p, and the result follows by induction.

Rubric:
(Partial)
1. Proved that k should be even.
2. Used the induction on $n+1$ and considered the equation for $\frac{n+1}{p}$.
3. Stated the identity $n+1 \mid \sum_{i=0}^{n}\left((-1)^i\binom{n}{i}\right)^k$ without proof.

(Almost)
1. Solution is almost complete, but made minor mistakes which are not negligible, such as coming up with a proof that only works for odd primes.

(II) Masked IMO-ProofBench

Problem: Same as (I) above.

Masked Proof:
Answer: all even k. Let $S(n) := \sum_{i=0}^{n}\binom{n}{i}^k$.
Necessity. Choose $n = 2$. We need $3 \mid S(2) = 2 + 2^k$, which requires k to be even.
Sufficiency. From now on treat k as fixed and even. Let p^e be a prime power fully dividing $n+1$. It suffices to prove $S(n)$ is a multiple of p^e.
Lemma. Suppose p^e fully divides $n+1$. Then

$$\binom{n}{i} \equiv \pm\binom{\frac{n+1}{p}-1}{\lfloor i/p \rfloor} \pmod{p^e}$$

Proof. Write $\binom{n}{i} = \prod_{1\le j\le i}\frac{n-(j-1)}{j}$. The fractions for $p \nmid j$ are all $\pm 1 \pmod{p^e}$. So only consider those j with $p \mid j$; in that case one obtains the claimed binomial exactly.
From the lemma:

[MASK]

by grouping the $n+1$ terms into consecutive ranges of length p, and the result follows by induction.

Masked content: $S(n) \equiv p \cdot S\left(\frac{n+1}{p} - 1\right) \pmod{p^e}$

(III) Masked Research Level Proof

Question Based on **Lemma 2.4** from Shankar et al. (2025):
Let K be a field of characteristic not 2. Let $f(x,y) \in U(K)$ be an element with nonzero discriminant. Then there exist unique $\mathrm{SL}_3(K)$-orbits on $\mathrm{Res}^{-1}(K) \cap W(K)$ which are distinguished and Δ-distinguished. The same result holds when $W(K)$ is replaced with $W^\vee(K)$.
Additional information (for self-containedness). The following material from the paper is included so the proof is self-contained.

- **Distinguished and Δ-distinguished orbits:** Assume without loss of generality that $a > 0$. We start by defining the notions of distinguished and Δ-distinguished orbits.
- **Definition 2.3** Let K be a field of characteristic not 2, and let $f \in U(K)$ of nonzero discriminant. We say that an $\mathrm{SL}_3(K)$-orbit on $\mathrm{Res}^{-1}(f) \cap W(K)$ is *distinguished* if it contains a representative (A,B) with $a_{11} = b_{11} = 0$; and Δ-*distinguished* if it contains a representative (A,B) with $a_{11}a_{22} - a_{12}^2 = b_{11}b_{22} - b_{12}^2 = 0$. Here, we denote by m_{ij} the ij-matrix entry of $M \in W(K)$ if $i = j$ and twice the ij-matrix entry if $i \neq j$.

Masked Proof:

Proof. We prove the result for $W(K)$; the proof for $W^\vee(K)$ is identical. Denote the coefficients of f by a, b, c, and d, and assume (by replacing f with a $\mathrm{GL}_2(K)$-translate, if necessary) that $ad \neq 0$.
First, it is easy to check that there exists a distinguished orbit (resp. a Δ-distinguished orbit) in $W(K)$. The distinguished orbit is represented by the following pair:

$$(A,B) = \left(\begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & -a & 0 \\ \frac{1}{2} & 0 & -c \end{pmatrix}, \begin{pmatrix} 0 & \frac{1}{2} & 0 \\ \frac{1}{2} & b & 0 \\ 0 & 0 & d \end{pmatrix}\right).$$

The Δ-distinguished orbit with resolvent f is represented by the following pair:

[MASK]

It only remains to prove uniqueness. For the distinguished case, let… (Too long to display, Full example is in Appendix E) □

Masked content:

$$(A,B) = \left(\begin{pmatrix} -a & 0 & 0 \\ 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{b}{4ad} \end{pmatrix}, \begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & d & 0 \\ \frac{1}{2} & 0 & \frac{-c}{4ad} \end{pmatrix}\right). \quad (1)$$

Example LLM Response (DeepSeek-V3.2-Thinking):

$$(A,B) = \left(\begin{pmatrix} -a & 0 & 0 \\ 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{b}{4ad} \end{pmatrix}, \begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & -d & 0 \\ \frac{1}{2} & 0 & \frac{c}{4ad} \end{pmatrix}\right). \quad (2)$$

Figure 1: (I) IMO-ProofBench [17] represents competition-level proof problems paired with full solutions and rubric-style evaluation. (II) After Mask-Proof processing, the same proof data are converted into a verifiable masked format, where key formula/derivation steps are hidden as targets for automatic evaluation. (III) Mask-ProofBench extends this masked, verifiable format to research-level proofs with substantially longer derivations, masking rigorous intermediate steps rather than final numerical results or highly templated fragments. See Appendix D for complete details.

whose closed-form definitions appear elsewhere); these are recovered by searching the source LaTeX for their defining statements and attaching the extracted content as Additional Information. The full prompt is provided in Appendix A. Cases dominated by external dependencies (e.g., citations to external conclusions) are discarded.

Agentic Masking for Proof. For each self-contained proof, we construct masked steps by replacing exactly one contiguous reasoning step with `[MASK]`. Each masked step contains only pure mathematical expressions (no natural-language text). From each source proof, Mask-Proof generate up to three such masked steps, each corresponding to an independent and well-defined inference. We

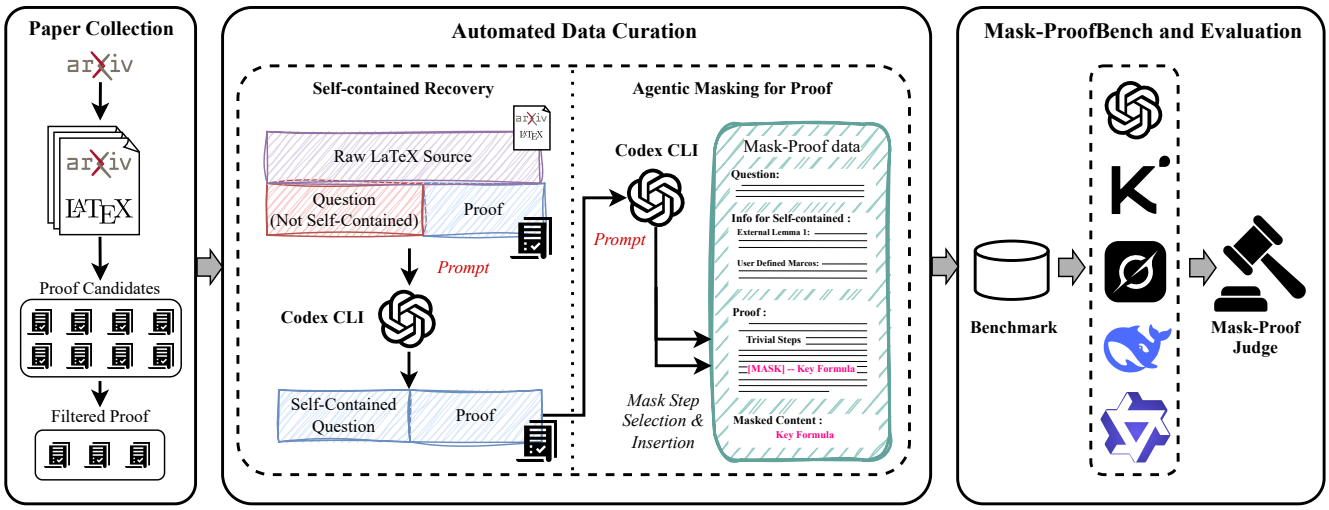

Figure 2: Mask-Proof Pipeline. Starting from raw arXiv LaTeX sources, our pipeline extracts complete proofs, repairs them into self-contained contexts, and agentically masks a key formula step to produce Mask-ProofBench.

defined a series of rubrics (see Appendix B) to guide the selection of masked positions. These rubrics instruct Codex CLI to agentically mask key inference-critical steps, ensuring that the resulting tasks necessarily require non-trivial mathematical reasoning rather than shallow pattern matching.

Faithful Mask Application. A practical failure mode of directly applying LLM-based masking is unintended proof rewriting, which can subtly alter semantics. To preserve faithfulness to the original proof, our pipeline decouples *(i) selecting mask steps* from *(ii) applying the mask*. We use Codex CLI to perform these two steps automatically under strict constraints, ensuring that only the target step is masked while the remaining proof text remains unchanged.

3.3 Automated Proof Data Evaluation

As illustrated in Figure 2, our automated proof-data curation pipeline converts each evaluation unit into a masked key formula step within a well-defined, self-contained proof context.

Compared to evaluating full proofs or sub-proofs, focusing on such mask instances substantially reduces ambiguity and leads to more consistent automated judgments.

Nevertheless, exact string matching remains inadequate for mathematical proofs due to extensive notational diversity and equivalent reformulations. To address this limitation, we introduce Mask-ProofJudge, an LLM-based evaluation framework where GPT-OSS-120B (High) serves as the backbone to assess semantic equivalence between model outputs and the ground-truth masked content (see Appendix A for the judging prompts).

To further reduce evaluation variance arising from generation stochasticity, we sample four solutions per problem and aggregate multiple independent judgments via majority voting, reporting the resulting average accuracy.

In practice, the LLM judge achieves a 96.8% agreement rate with human expert annotations, indicating that the proposed evaluation protocol provides a reliable and high-confidence approach for expert-level assessment.

3.4 Case Study

To explicitly demonstrate the workflow described in Sections 3.2 and 3.3, we trace the processing of *Lemma 2.4* from Shankar et al. [21]. This example, which corresponds to the case illustrated in Figure 1, shows how each stage of our pipeline transforms a raw arXiv fragment into a rigorous evaluation instance.

Stage 1: Self-contained Recovery. In this case, the raw extraction of *Lemma 2.4* requires constructing a matrix representation for the "A-distinguished orbit." However, the critical definition of "A-distinguished" is located in *Definition 2.3*, outside the local context [21]. As defined in our curation protocol, this constitutes an "unresolved external dependency." Our pipeline's dependency agent successfully identified this gap. Instead of discarding the proof, it traced the term back to *Definition 2.3*, extracted the invariant condition ($a_{11}a_{22} - a_{12}^2 = 0$), and appended it to the problem context. This step ensured the instance satisfied the self-contained criterion.

Stage 2: Agentic Mask Selection. Following the recovery, the Mask Selection Agent scanned the proof to identify a non-trivial reasoning step based on the rubrics in Appendix B. The agent rejected steps involving simple algebraic rearrangements (violating the "non-trivial" rubric) and selected the specific matrix construction step. This selection was strategic: correctly masking this matrix forces the model to synthesize the recovered global definition with local variables a, b, c, d. It ensures the task tests reasoning necessity rather than pattern matching or context copying.

Stage 3: Automated Evaluation. Finally, we evaluated the response from DeepSeek-V3.2-Thinking using Mask-ProofJudge. The model's output visually resembled the ground truth but inverted

the sign of the $(2, 2)$ entry from d to $-d$.This constitutes a mathematical failure, not a notational error.In orbital classification, the discriminant's sign is a structural invariant distinguishing disjoint $SL_3(K)$-orbits. By flipping the determinant (from $-d/4$ to $d/4$), the model constructed a representative from a different isomorphism class, violating the context-defined constraints. Mask-ProofJudge correctly identified this algebraic violation, marking the response as **Incorrect** (full invariant analysis in Appendix D).

4 Experiments

In this section, we evaluate Mask-Proof to ascertain its effectiveness in automated constructing masked problem following four questions: RQ1: How effective is Mask-Proof as an automated data curation pipeline? RQ2: How reproducible is the automated Mask-Proof pipeline? RQ3: Is our automated evaluation framework consistent with human experts in masked-step evaluation? RQ4: Can the Mask-Proof pipeline generalize to other data sources?

4.1 Experimental Setup

Dataset. To comprehensively evaluate the effectiveness of the Mask-Proof pipeline, we curate two datasets:

- **Mask-ProofBench**, constructed using the pipeline described in Section 3.2. We collect 403 arXiv papers spanning 35 mathematical fields, from which 835 theorem–proof blocks are extracted as raw candidates. This results in Mask-ProofBench with 292 masked instances (see Table 1 for a stage-wise breakdown).
- To evaluate the cross-dataset generalization of the automated pipeline, we additionally construct a comparison dataset by applying Mask-Proof to all 30 *PB-Advanced* proofs from IMO-ProofBench [17]. This comparison dataset consist of 41 mask instances.

Evaluation Metrics. We evaluate the Mask-Proof pipeline and the curated dataset using three complementary metrics: Avg@4, Spearman's rank correlation coefficient, and BLEU. Avg@4 measures the average correctness of an LLM over four independent attempts on each masked proof step. Spearman's rank correlation coefficient assesses the transferability of Mask-Proof by quantifying the consistency of model rankings of IMO-ProofBench under masked evaluation. BLEU evaluates n-gram overlap to measure surface-level linguistic consistency, and is used to assess the reproducibility of self-contained proof recovery. Together, these metrics characterize the effectiveness, transferability, and reproducibility of the Mask-Proof pipeline.

Evaluated Models. To conduct a comprehensive evaluation, we categorize state-of-the-art LLMs into two groups based on their reasoning capability (see Appendix E for details).

- **Standard models:** General-purpose models that generate answers directly without explicit deliberation (e.g., DeepSeek-V3.2, Grok-4.1, Qwen3-14B), serving as baselines to quantify the benefit of test-time reasoning.
- **Reasoning-enhanced models:** Models enhanced for multi-step thinking (e.g., DeepSeek-R1, Grok-4.1-reasoning, Gemini-3-Flash-preview), which produce internal chains of thought before outputting the final answer.

4.2 Effectiveness of the Automated Mask-Proof Pipeline (RQ1)

4.2.1 Main Results on Mask-ProofBench. We evaluate 17 models on **Mask-ProofBench**, including both reasoning and non-reasoning categories. Figure 3 presents the full leaderboard.

Reasoning Capability Creates a Qualitative Boundary. The results indicate that Mask-ProofBench clearly separates reasoning-enhanced models from standard models. A performance boundary emerges at approximately 32%: eight out of nine reasoning-enhanced models (except GPT-OSS-120B under low reasoning effort) exceed this threshold, whereas all six standard models fall below it. To further quantify this effect, we conduct paired comparisons within the same model families. For example, Grok-4.1-reasoning outperforms Grok-4.1 by 26.7%, demonstrating that the benchmark reliably and effectively captures improvements attributable to enhanced reasoning capability.

Highly discriminative, not saturated, and extensible. The leaderboard exhibits a wide performance range, from a lower bound of 8.1% (Qwen3-14B) to an upper bound of 63.3% (GPT-5.2 High) in Avg@4, indicating strong discriminative power. Notably, these results are obtained under a single-mask setting with our pipeline, where only one formula-level step is masked per proof. This design leaves substantial headroom for increasing benchmark difficulty, for example by masking multiple interdependent steps or requiring multi-step reconstruction as model capabilities advance.

No Anomalous Reversals. As a sanity check, we verify that the ranking contains no major surprises: smaller models do not outperform larger ones, and models known to struggle with mathematics do not unexpectedly excel. This coherence indicates that the benchmark measures a stable capability rather than task-specific artifacts.

4.2.2 Ablation Study: Effectiveness of the Self-Contained Module. The Mask-Proof pipeline intentionally resolves cross-references and expands user-defined macros. Manual inspection of all 292 masked instances in Mask-ProofBench confirms that 291 out of 292 are fully self-contained after this preprocessing.

To verify that our self-contained recovery agent effectively addresses the context incompleteness problem, we compare the performance of three models (Grok-4.1-reasoning, DeepSeek-V3.2-Thinking, and Grok-4.1) on a non–self-contained variant of Mask-ProofBench. Figure 5 shows consistent performance degradation under the non–self-contained variant of the benchmark. Model performance drops by 11% to 23%, with reasoning-enhanced models exhibiting larger declines, indicating that LLM performance in this evaluation task strongly depends on access to the original proof context provided by self-contained recovery. Notably, models still achieve moderate accuracy (13%–35%) without context resolution, suggesting that context incompleteness does not affect all masked instances. This is because even in non–self-contained proofs, not every masked step requires the missing original proof context for correct reconstruction. However, the non-negligible performance loss confirms that the self-containedness of research-level mathematical proofs affects a substantial portion of the benchmark, and that our dependency resolution effectively mitigates this issue.

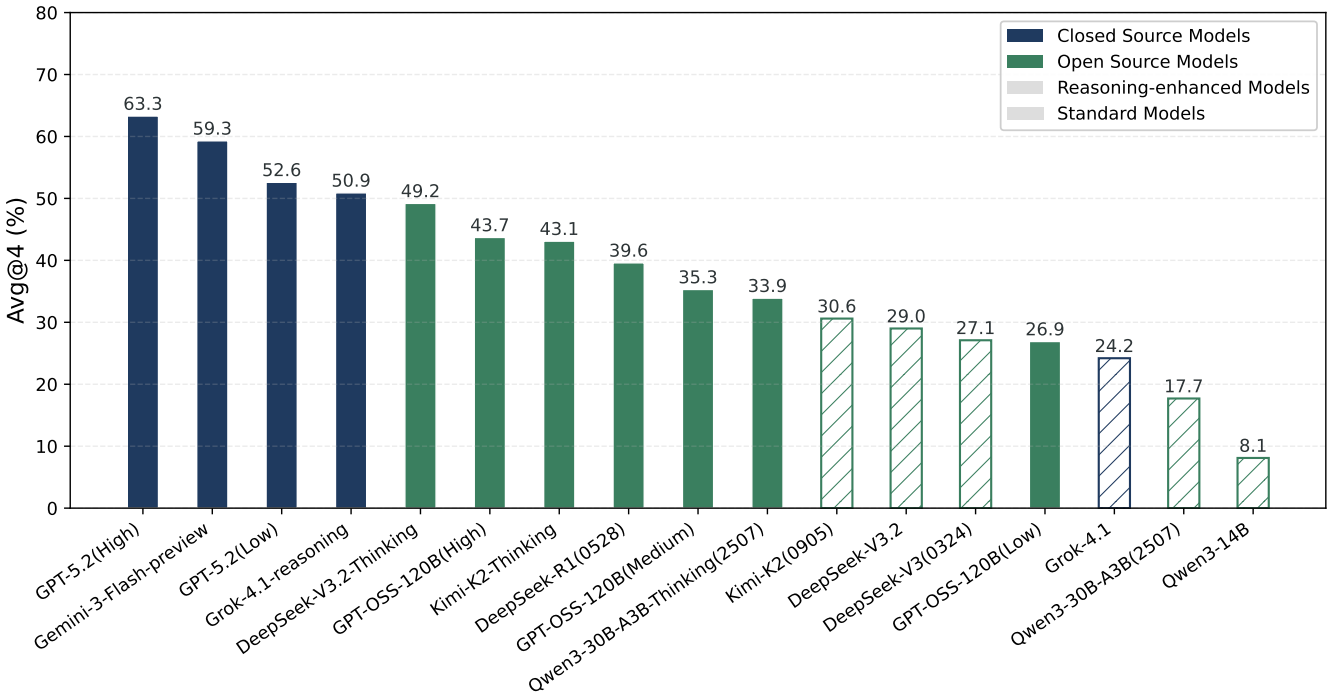

Figure 3: Avg@4 leaderboard on Mask-ProofBench. Discriminative power of Mask-ProofBench (Avg@4). Reasoning-enhanced models demonstrate a substantial performance advantage over standard models at comparable parameter scales.

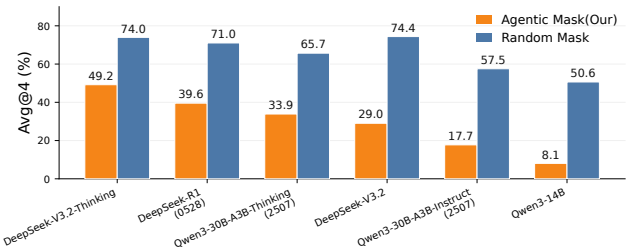

Figure 4: Leaderboard distortion from random masking. Avg@4 performance under *random* and *strategic* masking for representative models. Random masking substantially inflates scores—especially for standard models—while agentic masking consistently lowers performance and restores meaningful separation, demonstrating the necessity of anti-hack filtering for a discriminative benchmark.

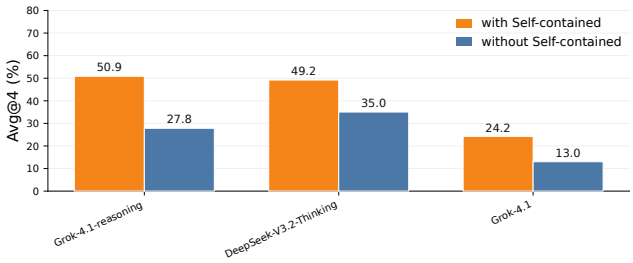

Figure 5: Self-containedness ablation. Removing dependency resolution causes 11%–23% performance descending, confirming that context completeness is essential for valid evaluation.

4.2.3 Ablation Study: Effectiveness of Our Agentic-Masking. A valid instance of mask-in-the-context problem must resist shortcut solutions. If models can achieve high scores by exploiting superficial cues(e.g. positional patterns, local context copying), the evaluation fails to measure genuine reasoning. We validate our mask instances by comparing against random step selection.

agentic masking Enables Reliable Model Differentiation. To evaluate the effectiveness of our agentic masking strategy, we introduce a random-masking baseline. The only difference between random masking and Mask-ProofBench is that the former selects mask steps uniformly at random, without any position-selection preferences. By comparing LLM performance on Mask-ProofBench and this baseline, we show that our mask selection approach leads to more reliable differentiation of models based on their reasoning capability.

Figure 4 shows that random masking substantially inflates Avg@4 across models. For instance, DeepSeek-V3.2 rises from 29.0% to 74.4% (+45.4%), and Qwen3-14B from 8.1% to 50.6% (+42.5%). Crucially, the score inflation caused by random masking is highly

asymmetric. Standard Models gain disproportionately more, leading to a strong compression of the reasoning advantage observed under agentic masking. In the DeepSeek family, agentic masking yields a clear performance separation, with DeepSeek-V3.2-Thinking achieving 49.2% and DeepSeek-V3.2 achieving 29.0%. Under random masking, both models reach nearly identical performance levels (74.0% vs. 74.4%), eliminating the gap observed under agentic masking. This suggests that a large fraction of randomly masked steps can be completed without true multi-step reasoning, relying instead on superficial pattern cues. Furthermore, random masking leads to unstable and misleading model rankings, where weaker standard models can match or even surpass stronger reasoning-enhanced models, contradicting established capability hierarchies.

4.3 Reproducibility of the Mask-Proof Pipeline (RQ2)

4.3.1 Self-Contained Recovery Reproducibility. To evaluate the reproducibility of our self-contained module, we run the pipeline twice independently on 20 sampled proofs from Mask-ProofBench. Three of these proofs are intrinsically self-contained and require no additional context.

Explicit dependencies. For references with explicit labels (via `\ref`, `\eqref`), we directly compare the resolved label sets. The two runs achieve 97.2% recall: out of 36 unique labels across both runs, 35 appear in both. The single discrepancy is an optional theorem reference included by one run but not the other. The high BLEU scores indicate that independently recovered Additional Information blocks are nearly identical across runs, suggesting that the implicit dependency resolution process is stable rather than sensitive to stochastic variation. **Implicit dependencies.** For implicit dependencies—such as symbols with closed-form definitions elsewhere—no ground-truth labels exist for direct comparison. We use textual similarity as a proxy: if two runs resolve the same implicit dependencies, they should produce similar Additional Information blocks. The symmetric BLEU-4 achieves a mean of 0.900 and median of 0.974, with all 20 proofs exceeding the 0.3 threshold.

4.3.2 Mask Selection Reproducibility. To evaluate the reproducibility of mask selection, we run the agentic masking module twice independently on the same 20 proofs used in Section 4.3.1. Both runs produce 28 masks in total. We measure reproducibility by checking how many masks from the first run appear in the second.

After normalizing each masked formula—stripping outer math delimiters (e.g., `$...$`, `\[...\]`) and whitespace—we check for exact matches within the same proof: 19 out of 28 masks (67.9%) are reproduced. We manually annotate all 9 non-reproduced masks into three categories:

- **Semantically proximate (6/9):** The two runs select different points along the same formula chain, e.g., an intermediate equation vs. the final result.
- **Same proof, different sub-step (1/9):** Both masks address related reasoning but test different skills, e.g., an error bound vs. the parameter choice that optimizes it.
- **Genuinely different (2/9):** The masks target unrelated mathematical content.

Counting semantically proximate cases as successful reproduction, the consistency rate rises from 67.9% to 89.3%. Only 2 out of 28 masks (7.1%) target genuinely different content.

Taken together, these results answer RQ2 by showing that the Mask-Proof pipeline is reproducible at both the self-contained recovery and mask selection stages. Independent runs produce highly consistent self-contained contexts and largely overlapping masks, with most discrepancies reflecting minor semantic variations rather than genuinely different problem constructions.

4.4 Consistency Between Human Experts and Mask-ProofJudge (RQ3)

Since our benchmark masks pure LaTeX mathematical formulas—where a single ground truth may have multiple semantically equivalent representations (e.g., notation variants, unsimplified terms)—we validate that Mask-ProofJudge can reliably assess equivalence.

We constructed a validation set of 1273 samples randomly drawn from actual responses of four different models(Gemini-3-Flash-preview, DeepSeek-V3.2-Thinking, Kimi-K2 and Grok-4.1), containing a balanced mix of correct answers and plausible but incorrect formulas. Two domain experts independently labeled the set by checking mathematical equivalence against the ground truth. Comparing Mask-ProofJudge's majority-vote verdicts to expert consensus, GPT-OSS-120B(High) achieves **96.8% accuracy**, indicating that it reliably distinguishes benign notational variation from substantive mathematical errors.

4.5 Cross-Dataset Generalization Ability (RQ4)

To evaluate whether the Mask-Proof pipeline generalizes beyond our curated benchmark, we apply it to an external dataset with existing expert evaluations, enabling direct comparison between our automated mask-based scoring and human expert judgments.

We apply the Mask-Proof pipeline to all 30 PB-Advanced proofs from IMO-ProofBench [17], producing 35 masks across three sources: Novel (22), IMO2024 (8), and USAMO2025 (5). We evaluate seven models using both our mask-based Avg@4 metric and the official expert scores. Table 2 presents the source-level breakdown and ranking consistency measured by Spearman's ρ.

The mask-based evaluation achieves Spearman $\rho = 0.79$ overall, reaching 0.89 on USAMO2025 and 0.86 on Novel. This correlation is obtained entirely automatically—no human grading, no problem-specific rubrics, and no per-task scoring guidelines—whereas IMO-ProofBench [17] expert evaluation requires domain experts to design rubrics and manually grade every response. Moreover, Mask-ProofJudge (backed by GPT-OSS-120B High), a mid-tier open-source model rather than frontier models or human experts. Despite this modest resource requirement, the resulting model rankings closely track expert consensus, demonstrating that Mask-Proof's discriminative power comes primarily from agentic masking of key formula steps rather than expensive evaluation infrastructure.

The moderate correlation on IMO2024 ($\rho = 0.73$) reflects a granularity mismatch: expert evaluations consider holistic proof structure and award partial credit for incomplete solutions, while mask-based evaluation targets localized step-level reasoning. These two

Table 2: Source-level breakdown of our Mask-Proof (Avg@4, %) compared with the official expert evaluation (% of total points) on IMO-ProofBench-Advanced.

Model	Mask-Proof (this work): Avg@4 (%)				Expert (official): score (%)			
	Adv	Novel	IMO2024	USAMO2025	Adv	Novel	IMO2024	USAMO2025
Number of Problems	35	22	8	5	30	18	6	6
o3	52.9	44.3	53.1	**90.0**	**20.5**	15.1	4.8	**52.4**
Gemini 2.5 Pro	**58.6**	**55.7**	46.9	**90.0**	17.6	**15.9**	**7.1**	33.3
Grok 4.1 Reasoning	50.0	43.2	**53.1**	75.0	18.6	19.8	16.7	16.7
o4-mini	42.1	33.0	46.9	75.0	11.4	8.7	7.1	23.8
Kimi-K2	30.7	23.9	25.0	70.0	7.1	4.0	2.4	21.4
Qwen3-235B	34.3	34.1	31.2	40.0	5.2	7.1	0.0	4.8
DeepSeek V3	32.9	30.7	25.0	55.0	4.3	6.3	2.4	0.0
Spearman ρ (Expert vs. Mask)	0.79	0.86	0.73	0.89			–	

paradigms are complementary—Mask-Proof excels at scalable, reproducible assessment of specific reasoning steps, while expert evaluation captures broader proof quality dimensions. Overall, Mask-Proof generalizes effectively to external proof sources and produces model rankings consistent with expert consensus, positioning it as a practical, scalable alternative for evaluating LLMs on research-level mathematical reasoning.

5 Conclusion

In this work, we make following contributions: **Mask-Proof**, an automatic pipeline for generating refreshable masked proof tasks and evaluating large language models on mathematical proofs. Mask-Proof generated **Mask-ProofBench**, a curated benchmark of 292 masked proof-step instances including 35 mathematical fields. Extensive experiments show that **Mask-Proof** can remove shortcut-solvable instances, and evaluations of **Mask-ProofBench** and Masked IMO-ProofBench reveal its effectiveness, reproducibility, and generalization across diverse proof sources.

6 Limitations and Ethical Considerations

Limitations. Our pipeline relies exclusively on LaTeX source, ensuring high data quality but limiting its use to documents with available source code. Additionally, the LLM-based judge (96.8% expert agreement) may still inherit biases from its underlying model.

Ethical Considerations. All source papers are drawn from arXiv under licenses permitting redistribution for research; we release only extracted proof fragments with attribution. To mitigate benchmark contamination, we restrict our corpus to the most recent three months of submissions and recommend periodic refresh. This work involves no human subjects.

References

[1] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. 2026. MathArena: Evaluating LLMs on Uncontaminated Math Competitions. arXiv:2505.23281 [cs.AI] https://arxiv.org/abs/2505.23281

[2] Sébastien Bubeck, Christian Coester, Ronen Eldan, Timothy Gowers, Yin Tat Lee, Alexandru Lupsasca, Mehtaab Sawhney, Robert Scherrer, Mark Sellke, Brian K. Spears, Derya Unutmaz, Kevin Weil, Steven Yin, and Nikita Zhivotovskiy. 2025. Early science acceleration experiments with GPT-5. arXiv:2511.16072 [cs.CL] https://arxiv.org/abs/2511.16072

[3] Guoxin Chen, Jing Wu, Xinjie Chen, Wayne Xin Zhao, Ruihua Song, Chengxi Li, Kai Fan, Dayiheng Liu, and Minpeng Liao. 2025. ReForm: Reflective Autoformalization with Prospective Bounded Sequence Optimization. arXiv:2510.24592 [cs.CL] https://arxiv.org/abs/2510.24592

[4] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *arXiv preprint arXiv:2211.12588* (2022).

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).

[6] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. 2025. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models. In *International Conference on Learning Representations (ICLR).* https://openreview.net/forum?id=yaqPf0KAlN

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2, 11 (Nov. 2020), 665–673. doi:10.1038/s42256-020-00257-z

[8] Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, Olli Järviniemi, Matthew Barnett, Robert Sandler, Matej Vrzala, Jaime Sevilla, Qiuyu Ren, Elizabeth Pratt, Lionel Levine, Grant Barkley, Natalie Stewart, Bogdan Grechuk, Tetiana Grechuk, Shreepranav Varma Enugandla, and Mark Wildon. 2024. FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI. arXiv:2411.04872 [cs.AI] https://arxiv.org/abs/2411.04872

[9] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).* Association for Computational Linguistics, Bangkok, Thailand, 3828–3850. doi:10.18653/v1/2024.acl-long.211

[10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *NeurIPS* (2021).

[11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations (ICLR).* https://openreview.net/forum?id=IkmD3fKBPQ

[12] Pei Ke, Bosi Wen, Zhuoer Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, Jie Tang, and Minlie Huang. 2024. CritiqueLLM: Towards an Informative Critique Generation Model for Evaluation of Large Language Model Generation. arXiv:2311.18702 [cs.CL] https://arxiv.org/abs/2311.18702

[13] Seungone Kim, Jamin Shin, Yejin Cho, et al. 2024. Prometheus: Inducing Fine-grained Evaluation Capability in Language Models. In *The Twelfth International Conference on Learning Representations.*

[14] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *International Conference on Learning Representations (ICLR).*

https://openreview.net/forum?id=v8L0pN6EOi

[15] Wenhao Liu, Zhenyi Lu, Xinyu Hu, Jerry Zhang, Dailin Li, Jiacheng Cen, Huilin Cao, Haiteng Wang, Yuhan Li, Xie Kun, Dandan Li, Pei Zhang, Chengbo Zhang, Yuxiang Ren, Xiaohong Huang, and Yan Ma. 2025. STORM-BORN: A Challenging Mathematical Derivations Dataset Curated via a Human-in-the-Loop Multi-Agent Framework. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 23938–23958. doi:10.18653/v1/2025.findings-acl.1227

[16] Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Meiqi Guo, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, and Abhinav Rastogi. 2024. Improve Mathematical Reasoning in Language Models by Automated Process Supervision. arXiv:2406.06592 [cs.CL] https://arxiv.org/abs/2406.06592

[17] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. 2025. Towards Robust Mathematical Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. https://aclanthology.org/2025.emnlp-main.1794/

[18] Wenjie Ma, Andrei Cojocaru, Neel Kolhe, Bradley Louie, Robin Said Sharif, Haihan Zhang, Vincent Zhuang, Matei Zaharia, and Sewon Min. 2025. Reliable Fine-Grained Evaluation of Natural Language Math Proofs. arXiv:2510.13888 [cs.CL] https://arxiv.org/abs/2510.13888

[19] Shrey Pandit, Austin Xu, Xuan-Phi Nguyen, Yifei Ming, Caiming Xiong, and Shafiq Joty. 2025. Hard2Verify: A Step-Level Verification Benchmark for Open-Ended Frontier Math. arXiv:2510.13744 [cs.AI] https://arxiv.org/abs/2510.13744

[20] Johannes Schmitt, Gergely Bérczi, Jasper Dekoninck, Jeremy Feusi, Tim Gehrunger, Raphael Appenzeller, Jim Bryan, Niklas Canova, Timo de Wolff, Filippo Gaia, Michel van Garrel, Baran Hashemi, David Holmes, Aitor Iribar Lopez, Victor Jaeck, Martina Jørgensen, Steven Kelk, Stefan Kuhlmann, Adam Kurpisz, Chiara Meroni, Ingmar Metzler, Martin Möller, Samuel Muñoz-Echániz, Robert Nowak, Georg Oberdieck, Daniel Platt, Dylan Possamaï, Gabriel Ribeiro, Raúl Sánchez Galán, Zheming Sun, Josef Teichmann, Richard P. Thomas, and Charles Vial. 2025. IMProofBench: Benchmarking AI on Research-Level Mathematical Proof Generation. arXiv:2509.26076 [cs.CL] https://arxiv.org/abs/2509.26076

[21] Arul Shankar, Artane Siad, and Ashvin A Swaminathan. 2025. Counting integral points on symmetric varieties with applications to arithmetic statistics. *Proceedings of the London Mathematical Society* 130, 4 (2025), e70039.

[22] Zhihong Shao, Yuxiang Luo, Chengda Lu, Z. Z. Ren, Jiewen Hu, Tian Ye, Zhibin Gou, Shirong Ma, and Xiaokang Zhang. 2025. DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning. arXiv:2511.22570 [cs.AI] https://arxiv.org/abs/2511.22570

[23] Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. 2025. Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges. arXiv:2406.12624 [cs.CL] https://arxiv.org/abs/2406.12624

[24] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. 2024. Solving olympiad geometry without human demonstrations. *Nature* 625, 7995 (2024), 476–482.

[25] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. 2024. PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. In *NeurIPS 2024 Track on Datasets and Benchmarks*. https://openreview.net/forum?id=ChKCF75Ocd

[26] Gladys Tyen, Hassan Mansoor, Victor Carbune, Peter Chen, and Tony Mak. 2024. LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13894–13908. doi:10.18653/v1/2024.findings-acl.826

[27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[28] Bo Yang, Qingping Yang, Yingwei Ma, and Runtao Liu. 2025. UTMath: Math Evaluation with Unit Test via Reasoning-to-Coding Thoughts. arXiv:2411.07240 [cs.CL] https://arxiv.org/abs/2411.07240

[29] Longhui Yu, Weisen Jiang, Han Shi, et al. 2023. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. *arXiv preprint arXiv:2309.12284* (2023).

[30] Jie Zhang, Cezara Petrui, Kristina Nikolić, and Florian Tramèr. 2025. RealMath: A Continuous Benchmark for Evaluating Language Models on Research-Level Mathematics. In *NeurIPS 2025 Track on Datasets and Benchmarks*. https://openreview.net/forum?id=RBssYVpQEr

[31] Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying Process Errors in Mathematical Reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 1009–1024. doi:10.18653/v1/2025.acl-long.50

[32] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. 2022. miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=9ZPegFuFTFv

[33] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2020, 29 pages.

[34] Kaijie Zhu, Jindong Wang, Jiaqi Zhou, et al. 2024. DyVal: Graph-informed Dynamic Evaluation of Large Language Models. In *The Twelfth International Conference on Learning Representations*.

A Prompts

This appendix documents the two system-level prompts used in our Mask-Proof pipeline, corresponding to two logically distinct but tightly coupled stages: (i) automated evaluation of masked proof steps(Table 3), and (ii) self-contained recovery(Table 4).

Table 3: Prompt template of automated proof data evaluation

System Prompt of Evaluation

You are an expert mathematician evaluating a 'fill-in-the-mask' task within a formal mathematical proof. Your goal is to determine if the Generated Answer (GA) is **Structurally Isomorphic** to the Ground Truth (GT).

Core Philosophy: "The State Machine Principle"

The Ground Truth (GT) defines the **Target State** for the current step. The Generated Answer (GA) is Correct ONLY if it aligns with the GT's state in Value, Stage, and Scope.

Pre-Evaluation Parsing Rule (Chain of Equalities)

IF the Generated Answer (GA) contains a chain of equalities (e.g., $A = B = C$) **AND** the Ground Truth (GT) is a single expression:

- You must isolate the **FINAL result** (the expression after the last equal sign) of the GA.
- Compare ONLY this final expression against the GT.
- *Example:* If GT is "2x" and GA is "x + x = 2x", evaluate "2x" vs "2x". This is CORRECT.

Evaluation Standard: The 3-Layer Filter

To be marked **Correct**, the (parsed) answer must pass ALL three layers:

Layer 1: Granularity & State Fidelity (The "Step" Check).

- **Rule:** If GT is in State N (e.g., Setup, Expansion, or Result), GA must be in State N.
- **Operator Conservation:** Major operators ($\sum$, $\int$, lim) present in the GT **must be preserved** in the GA unless trivial re-ordering.

Layer 2: Mathematical Equivalence (The Baseline).

- **Check:** Does the GA represent the exact same mathematical value, set, or proposition as the GT?

Layer 3: Allowable Deformations (Horizontal Movements).

- **Allowed (Isomorphic):** Factoring, Notation Styles (e^x vs $\exp(x)$), Commutativity, Variable Renaming.
- **Forbidden (Structural Alterations):** Term Merging (GT: $A + B$, GA: C), Definition Expansion.

Supplemental Rule: Long GT Compression & Target Extraction (Minimal Add-on)

- The Ground Truth (GT) may contain a long chain of equalities whose purpose is to derive a single final expression.
- You must treat the GT as defining a single **Target Expression** corresponding to its final logical state.
- The Generated Answer (GA) is NOT required to reproduce intermediate transformations (e.g., integration by parts, boundary term cancellation, divergence theorem applications) if these steps are already resolved in the GT.

Target Extraction Rule (GT-side)

- If the GT contains multiple equalities or derivation steps, you must conceptually extract the **final resolved expression** that represents the target state.
- Comparison with GA should be performed against this extracted target, not against the full derivation length.

Length Mismatch Clarification

- A shorter GA can still be correct if it directly matches the GT's final target state.
- Differences in derivation length, verbosity, or omission of already-eliminated terms MUST NOT be treated as incorrectness.

Final Decision Constraint

- If GA directly expresses the same final mathematical object that GT reaches at the end of its derivation, then GA is CORRECT, provided Layers 1–3 are satisfied.

Output Format

Return ONLY a JSON object:

{ "is_correct": boolean, "explanation": "1. Parsing (Did you strip equality chain?). 2. Math Equivalence. 3. Logical State. 4. Deformations." }

Table 4: Prompt template of self-contained recovery

System Prompt of Self-contained Recovery

Task Description. You are given a JSONL file containing extracted theorem–proof pairs: `{ File Path }`.
You are also given the full LaTeX sources of the corresponding papers, located at: `{ Original Paper LaTeX file Path }`.
Your task is to process each extracted proof and transform it into a *self-contained* proof, using the full paper LaTeX sources as reference when necessary.
The final output must be written to a new JSONL file. If the target output file already exists, *do not overwrite it*; instead, append the new entries to the end of the file.

Core Objective. For each theorem–proof pair:

- Ensure that the proof can be fully understood and completed *without referring to any external material* other than the problem statement itself.
- Any additional information required to make the proof self-contained must be added to the `"question"` field as *Additional Information*.

Self-Containedness Criteria. A proof is considered *not self-contained* if any of the following issues occur:

(1) **Missing theorem or lemma statements.** If the proof refers to a theorem, lemma, or proposition whose statement is not included, locate the corresponding statement in the original paper using its LaTeX label and append it to the `"question"` field.
(2) **Missing key definitions with explicit mathematical content.** If the proof relies on a symbol or object whose explicit definition (e.g. a closed-form expression or precise construction) is given elsewhere in the paper, and that definition is essential for the reasoning, append the definition to the `"question"` field.
(3) **Non-standard or paper-specific symbol meanings.** Some symbols may be defined in a non-standard or paper-specific way. You must decide whether the meaning of such a symbol can reasonably be assumed as common background knowledge for the solver, or whether it must be explicitly provided. If explicit clarification is needed, append the definition to the `"question"` field.
(4) **Custom LaTeX macros.** Inspect each paper for user-defined macros. Any custom macros appearing in the proof must be replaced with equivalent standard LaTeX commands. The final proof text must not rely on paper-specific macro definitions.

Standardization Requirements. The standardized self-contained version must satisfy all of the following:

- All custom LaTeX macros are rewritten using standard LaTeX commands.
- The theorem statement and proof together form a complete, standalone mathematical problem.
- Any lemmas, definitions, or constructions used in the proof but defined elsewhere in the paper are explicitly included when necessary.
- For symbols with unconventional meanings, you must judge whether the definition is part of assumed background knowledge or must be explicitly stated. If the latter, include the definition.
- Beyond the listed cases, you are expected to use semantic and mathematical judgment to determine what additional information is required so that a solver can complete the proof by reading the problem alone.

Output Constraint. Only output the transformed JSON entries. Do not include explanations, commentary, or intermediate reasoning. Ensure the output JSON is valid and directly appendable to an existing JSONL file.

B Mask Rubrics.

Our mask rubrics are as follow:

- **Formula-level completeness.** Each mask must correspond to a *complete mathematical expression or formula*, rather than a fragment of an equation or a syntactic sub-component. Partial masking of symbols, coefficients, or operators is disallowed.
- **Global reasoning dependency.** A valid mask should not be solvable by inspecting only its immediate local context. Correct recovery must require integrating information from the theorem statement, earlier definitions or lemmas, and the global logical structure of the proof.
- **Reasoning density and necessity.** Masked steps should represent logically critical steps (e.g., non-trivial formulas, structural identities, or key transformations) whose absence would break the proof. Trivial algebraic rearrangements or routine substitutions are excluded.
- **Non-templated and non-heuristic.** Masks must not be recoverable via pattern matching, standard proof templates, symmetry guessing, backward inference from later lines, or memorized formula instantiation. Any mask that can be solved without genuine mathematical reasoning is considered invalid.
- **Quality-controlled multiplicity.** For each proof, we allow between 0 and 3 masks, determined by the proof's conceptual depth, technical sophistication, and reasoning density. Proofs that are low-quality, overly short, informal, or mathematically trivial may yield zero masks.

C Data format of Mask.

Data format. We serialize the processed samples in `jsonl`. Each record contains a unique identifier and the paired question/solution:

```
{
"index": 6, "arxiv_id": "...",
```

```
"proof_index": 8,
"problem": <<PROBLEM TO PROVE>>,
"reference_solution": <<PROOF>>,
"mask_text": <<PROOF CONTAINS 1 MASK>>,
"mask_content": <<ORIGINAL TEXT IN THE MASK>>
}
```

Here `mask_text` is the masked prompt (typically formula-only masks), and `mask_content` is the corresponding ground-truth proof fragment.

D Case Study

In Section 3.4, we provided a high-level overview of the pipeline's handling of Lemma 2.4 from Shankar et al. [21]. This appendix provides a granular analysis of the input data processing, agentic mask selection, and a rigorous mathematical derivation of the model's failure mode, referencing the full case details presented in **Figure 6**.

D.1 Context Recovery Analysis

As observed in the **"Raw Proof Segment"** (see Figure 6), the proof's logic hinges entirely on the concept of a *"Δ-distinguished orbit."* However, this term is defined in **Definition 2.3**, which lies outside the local context of Lemma 2.4.

Without this definition, the matrix construction is mathematically indeterminate. There are infinite matrices (A, B) that could represent an orbit, but only those satisfying the algebraic constraint $a_{11}a_{22} - a_{12}^2 = 0$ (from Definition 2.3) are valid candidates. Our Dependency Agent correctly identified this semantic gap. By tracing the term "Δ-distinguished" backward through the document tree, it retrieved the definition and injected it into the **"Additional Information"** field. This intervention transformed an unsolvable fragment into a self-contained reasoning task.

D.2 Mask Selection Rationale

The Mask Selection Agent targeted the explicit matrix construction step:

$$B = \begin{pmatrix} 0 & 0 & 1/2 \\ 0 & d & 0 \\ 1/2 & 0 & -c/4ad \end{pmatrix}$$

This selection aligns with our rubric (Appendix B) for two reasons:

- **Synthesizing Global and Local Constraints:** Recovering this matrix requires the model to apply the retrieved global definition ($a_{11}a_{22} - a_{12}^2 = 0$) to the local variables a, b, c, d. It forces the model to perform variable substitution rather than simple text completion.
- **Avoiding Triviality:** The agent rejected earlier steps in the proof (such as the assumption $a_{12} = 0$) because they were simple algebraic simplifications that did not test deep comprehension of the orbital properties.

D.3 Mathematical Forensic: The Sign Error

We evaluated **DeepSeek-V3.2-Thinking** on this instance. As shown in the "Example LLM Response" in Figure 6, the model generated a matrix structure B_{Model} that visually resembles the Ground Truth B_{GT}, but with sign inversions at entries $(2, 2)$ and $(3, 3)$.

While a naive evaluator might dismiss this as a formatting nuance, Mask-ProofJudge correctly flagged it as a fundamental mathematical error. We justify this verdict via **Invariant Theory**:

In the context of symmetric varieties, a necessary condition for two matrices to represent the same orbit is that they must preserve the discriminant invariant. We calculate the determinant of matrix B for both cases:

Ground Truth Invariant (B_{GT}):

$$\det(B_{GT}) = d \cdot \left((0)(0) - \left(\frac{1}{2}\right)\left(\frac{1}{2}\right)\right) = -\frac{\mathbf{d}}{\mathbf{4}}$$

Model Output Invariant (B_{Model}): The model set $b_{22} = -d$. Expanding the determinant:

$$\det(B_{Model}) = -d \cdot \left((0)(0) - \left(\frac{1}{2}\right)\left(\frac{1}{2}\right)\right) = +\frac{\mathbf{d}}{\mathbf{4}}$$

Conclusion. Since the field characteristic is not 2 (explicitly stated in the retrieved Definition 2.3), we have $-\frac{d}{4} \neq \frac{d}{4}$. The sign flip implies that the model constructed a representative from a **disjoint isomorphism class** (effectively, a different orbit). Mask-ProofJudge's refusal to accept this answer demonstrates the pipeline's ability to distinguish between valid notational deformations and invalid algebraic violations.

Case Study: Δ-Distinguished Orbit (Lemma 2.4)

Question Based on **Lemma 2.4** from [21]:
Let K be a field of characteristic not 2. Let $f(x, y) \in U(K)$ be an element with nonzero discriminant. Then there exist unique $\mathrm{SL}_3(K)$-orbits on $\mathrm{Res}^{-1}(K) \cap W(K)$ which are distinguished and Δ-distinguished. The same result holds when $W(K)$ is replaced with $W^\vee(K)$.
Additional information (for self-containedness). The following material from the paper is included so the proof is self-contained.
- **Distinguished and Δ-distinguished orbits:** Assume without loss of generality that $a > 0$. We start by defining the notions of distinguished and Δ-distinguished orbits.
- **Definition 2.3** Let K be a field of characteristic not 2, and let $f \in U(K)$ of nonzero discriminant. We say that an $\mathrm{SL}_3(K)$-orbit on $\mathrm{Res}^{-1}(f) \cap W(K)$ is *distinguished* if it contains a representative (A, B) with $a_{11} = b_{11} = 0$; and Δ-*distinguished* if it contains a representative (A, B) with $a_{11}a_{22} - a_{12}^2 = b_{11}b_{22} - b_{12}^2 = 0$. Here, we denote by m_{ij} the ij-matrix entry of $M \in W(K)$ if $i = j$ and twice the ij-matrix entry if $i \neq j$.

Masked Proof:

Proof. We prove the result for $W(K)$; the proof for $W^\vee(K)$ is identical. Denote the coefficients of f by a, b, c, and d, and assume (by replacing f with a $\mathrm{GL}_2(K)$-translate, if necessary) that $ad \neq 0$.
First, it is easy to check that there exists a distinguished orbit (resp. a Δ-distinguished orbit) in $W(K)$. The distinguished orbit is represented by the following pair:

$$(A, B) = \left(\begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & -a & 0 \\ \frac{1}{2} & 0 & -c \end{pmatrix}, \begin{pmatrix} 0 & \frac{1}{2} & 0 \\ \frac{1}{2} & b & 0 \\ 0 & 0 & d \end{pmatrix}\right).$$

The Δ-distinguished orbit with resolvent f is represented by the following pair:

[MASK]

It only remains to prove uniqueness. For the distinguished case, let L be the étale cubic extension of K corresponding to f under the Delone–Faddeev bijection. $\mathrm{SL}_3(K)$-orbits on $W(K) \cap \mathrm{Res}^{-1}(f)$ correspond bijectively to étale quartic extensions of K with resolvent f by Bhargava's parametrization. An orbit is distinguished if and only if it corresponds to $L \oplus K$, and so there is a unique distinguished orbit.
We give a much more hands-on proof of uniqueness of the Δ-distinguished orbit. Let (A, B) (with coefficients a_{ij} and b_{ij}) be an element in a Δ-distinguished orbit with $\mathrm{Res}(A, B) = f$. By the definition of being Δ-distinguished, there exists a $\mathbb{P}^1_K \subset \mathbb{P}^2_K$ such that the restriction of A and B to this $\mathbb{P}^1_K$ give reducible binary quadratic forms. (Note that neither of these binary forms can be 0, otherwise we would have $\Delta(A, B) = 0$.)
By replacing (A, B) with an $\mathrm{SL}_3(K)$-translate, we may assume that $\mathbb{P}^1_K$ is the subspace $(*, *, 0)$, and we may hence assume that $a_{12} = a_{22} = b_{11} = b_{12} = 0$. Since a_{11} and b_{22} are nonzero, we may use another $\mathrm{SL}_3(K)$-transformation to further assume that $a_{13} = b_{23} = 0$. Then we have

$$a/4 = \det(A) = -a_{11}a_{23}^2 \quad \text{and} \quad b/4 = \det(B) = -b_{22}b_{13}^2.$$

Thus, with another $\mathrm{SL}_3(K)$-transformation, we can ensure that $a_{11} = -a$, $b_{22} = -d$, and $a_{23} = b_{13} = \frac{1}{2}$. Finally, the coefficients a_{33} and b_{33} are uniquely determined by the values of b and c. The lemma follows. □

Masked content:

$$(A, B) = \left(\begin{pmatrix} -a & 0 & 0 \\ 0 & 0 & 1/2 \\ 0 & 1/2 & b/4ad \end{pmatrix}, \begin{pmatrix} 0 & 0 & 1/2 \\ 0 & d & 0 \\ 1/2 & 0 & -c/4ad \end{pmatrix}\right). \tag{1}$$

Example LLM Response:

$$(A, B) = \left(\begin{pmatrix} -a & 0 & 0 \\ 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{b}{4ad} \end{pmatrix}, \begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & -d & 0 \\ \frac{1}{2} & 0 & \frac{c}{4ad} \end{pmatrix}\right). \tag{2}$$

Mask-ProofJudge: GA is Wrong.
Explaination: 1. Math Equivalence: The generated matrices differ from the ground-truth in sign for two entries: the (2,2) entry of the second matrix is -d instead of d, and the (3,3) entry is +c/(4ad) instead of -c/(4ad). Hence the expressions are not mathematically equal. 2. Logical State: Both answers are in the same proof step (the Δ-distinguished orbit representation), so the state matches. 3. Deformations: The sign changes are not allowed admissible deformations (they are not simple re-ordering, factoring, or variable renaming). Therefore the generated answer is not structurally isomorphic to the ground truth.

Figure 6: A detailed view of the Lemma 2.4 case study. The box illustrates the specific context injected by the agent, the masked derivation step, and the comparison between the Ground Truth and the Model's erroneous output.

E Detail of evaluated models

Table 5 shows the model name, open source status, number of parameters, reasoning ability, and the **Avg@4** accuracy on the Mask-ProofBench.

Model	Source	Reasoning	Avg@4
GPT-5.2 (High)	Closed	✓	63.3%
GPT-5.2 (Low)	Closed	✓	52.6%
GPT-oss-120b (High)	Open	✓	43.7%
GPT-oss-120b (Medium)	Open	✓	35.3%
GPT-oss-120b (Low)	Open	✓	26.9%
Gemini-3-Flash-Preview	Closed	✓	59.3%
Grok 4.1-reasoning	Closed	✓	50.9%
Grok 4.1	Closed	×	24.2%
DeepSeek-V3.2 Thinking	Open	✓	49.2%
DeepSeek-V3.2	Open	×	29.0%
DeepSeek-R1(0528)	Open	✓	36.9%
DeepSeek-V3(0324)	Open	×	27.1%
Qwen3-30B-A3B-Thinking	Open	✓	33.9%
Qwen3-30B-A3B	Open	×	17.7%
Qwen3-14B	Open	×	8.1%
Moonshot Kimi-K2-Thinking	Open	✓	43.1%
Moonshot Kimi-K2(0905)	Open	×	30.6%

Table 5: Evaluated models and their configurations. Reasoning-enhanced variants allow additional internal computation at inference time, while standard variants use default decoding settings. All models are evaluated under the same sampling budget ($k = 4$).

F Data Distribution

Table 6: Percentage distribution of the top arXiv subject categories of papers used in Mask-ProofBench.

Category	Percentage	Category	Percentage
PDE / Analysis	26.47%	Combinatorics	2.94%
Mathematical Physics	5.88%	Numerical Analysis	2.94%
Number Theory	5.15%	Optimization and Control	2.94%
Differential Geometry	5.15%	Operator Algebras y	2.94%
Functional Analysis	4.41%	Algebraic Geometry	2.21%
Representation Theory	4.41%	Algebraic Topology	2.21%
Probability	3.68%	Statistics Theory	2.21%
Classical Analysis and ODEs	2.94%	Others	23.52%